

CAS PRATIQUE

Sécurité Réseau avec Python

Reniflage, Empoisonnement DNS et ARP Spoofing

Prérequis	Python de base, protocoles TCP/IP, modèle OSI
Environnement	Laboratoire isolé — Machines virtuelles uniquement
Travaux	TP1 — TP2 — TP3

Document à usage exclusivement pédagogique.
Toute utilisation hors environnement de laboratoire contrôlé est interdite.

Avertissement légal et éthique

Les manipulations décrites dans ce document sont réservées à un cadre pédagogique strict. Elles doivent impérativement être effectuées dans un environnement de laboratoire entièrement isolé, sur des machines virtuelles appartenant à l'étudiant. Toute reproduction de ces techniques sur un réseau tiers, sans autorisation explicite et écrite du propriétaire du système, est constitutive d'une infraction pénale dans la majorité des juridictions : accès frauduleux à un système de traitement automatisé de données, interception de communications électroniques, usurpation d'identité numérique.

L'objectif de ces travaux est de comprendre les mécanismes d'attaque afin de concevoir des architectures défensives robustes et de justifier les bonnes pratiques de sécurité auprès des utilisateurs et des décideurs.

Architecture du laboratoire

Les trois travaux pratiques reposent sur la topologie réseau suivante. L'ensemble des machines doit être placé dans un réseau virtuel privé, sans aucun accès à Internet durant les manipulations.

Machine	Rôle	OS suggéré	Adresse IP
Attaquant	Poste de l'étudiant	Kali Linux	192.168.100.10
Victime	Cible des démonstrations	Debian 12 ou Windows 11	192.168.100.20
Passerelle	Routeur du laboratoire	Toute distribution Linux	192.168.100.1

Aucune interface de la machine attaquante ne doit disposer d'un accès Internet actif pendant les manipulations.

Table des matières

1	TP1 — Renifleur de Réseau et Capture de Credentials en Clair	4
1.1	Contexte	4
1.2	Environnement à préparer	4
1.3	Partie A — Renifleur générique	5
1.3.1	Questions de compréhension — Partie A	5
1.4	Partie B — Module d'extraction de credentials	5
1.4.1	Questions de compréhension — Partie B	6
2	TP2 — Empoisonnement DNS (DNS Spoofing)	7
2.1	Contexte	7
2.2	Rappels protocolaires	7
2.3	Partie A — Analyseur DNS passif	8
2.4	Partie B — Implémentation de l'empoisonnement DNS	8
2.5	Partie C — Validation et démonstration	9
2.5.1	Questions de compréhension	9
3	TP3 — ARP Spoofing et Redirection de Trafic avec iptables	10
3.1	Contexte	10
3.2	Rappels protocolaires	10
3.2.1	Champs d'un paquet ARP	10
3.2.2	Principe de l'attaque bidirectionnelle	11
3.3	Partie A — Relevé de l'état initial	11
3.4	Partie B — Empoisonneur ARP bidirectionnel	11
3.5	Partie C — Activation du transfert IP	12
3.6	Partie D — Redirection de trafic avec iptables	12
3.7	Partie E — Extension au contexte Wi-Fi	13
3.7.1	Questions de compréhension générales	13

1. TP1 — Renifleur de Réseau et Capture de Credentials en Clair

1.1. Contexte

De nombreux protocoles historiques transmettent les données d'authentification sans aucun chiffrement. C'est le cas de FTP, Telnet, POP3, SMTP en mode non sécurisé et HTTP en dehors de toute couche TLS. Un attaquant positionné passivement sur le même segment réseau peut capturer ces informations sans émettre le moindre paquet suspect, rendant l'attaque particulièrement difficile à détecter.

Ce travail pratique vous amène à implémenter un renifleur de paquets en Python capable d'identifier et d'extraire des identifiants transmis en clair, afin de comprendre concrètement pourquoi le chiffrement des communications est une exigence de sécurité non négociable.

Objectifs pédagogiques

- Comprendre le principe de la capture de paquets passive sur un segment réseau local.
- Maîtriser une bibliothèque Python de manipulation de paquets réseau pour l'analyse et la dissection de protocoles.
- Savoir écrire et appliquer un filtre BPF (Berkeley Packet Filter) ciblé sur des protocoles spécifiques.
- Extraire des informations sensibles depuis des flux protocolaires non chiffrés : FTP, HTTP, Telnet.
- Sauvegarder les captures au format `.pcap` exploitable sous Wireshark.

1.2. Environnement à préparer

Avant de commencer, vérifiez que les éléments suivants sont en place.

- Les bibliothèques Python nécessaires à la manipulation de paquets réseau sont installées sur la machine attaquante.
- Un serveur FTP (`vsftpd`) est actif sur la machine victime avec au moins un compte utilisateur de test.
- Un serveur Telnet (`telnetd`) est actif sur la machine victime avec un compte de test distinct.
- Un serveur HTTP (port 80, sans TLS) est accessible sur la machine victime et présente une page avec un formulaire de connexion comportant les champs `username` et `password`.
- Les deux machines se voient mutuellement via `ping`.

1.3. Partie A — Renifleur générique

Travail demandé

Écrivez un script Python nommé `sniffer.py` répondant aux spécifications suivantes.

1. Le script accepte en arguments depuis la ligne de commande le nom de l'interface réseau à écouter et un filtre BPF optionnel passé entre guillemets.
2. Pour chaque paquet capturé, le script affiche en console les informations suivantes lorsqu'elles sont disponibles : l'horodatage précis, le protocole identifié (TCP, UDP ou ICMP), l'adresse IP source, l'adresse IP destination, le port source et le port destination.
3. Le script écrit simultanément les paquets bruts dans un fichier `capture.pcap` lisible par Wireshark, et un résumé textuel dans un fichier `capture.log`.
4. Le programme se termine proprement sur interruption clavier sans lever d'exception non interceptée, et affiche un bilan final du nombre de paquets capturés par protocole.

1.3.1. Questions de compréhension — Partie A

Répondez par écrit aux questions suivantes avant de passer à la partie B.

1. Définissez le concept de filtre BPF. Donnez la syntaxe exacte permettant de capturer simultanément le trafic FTP (port 21) et HTTP (port 80).
2. Expliquez la différence entre une capture en mode promiscuité et une capture standard. Dans quel scénario le mode promiscuité est-il indispensable pour capturer le trafic d'une machine tierce sur un segment commuté ?
3. Pourquoi votre script nécessite-t-il les privilèges superutilisateur pour s'exécuter ? Répondez en vous appuyant sur le fonctionnement des sockets réseau au niveau du système d'exploitation Linux.

1.4. Partie B — Module d'extraction de credentials

Travail demandé

En vous appuyant sur le renifleur développé en partie A, étendez votre script pour extraire automatiquement les identifiants des protocoles suivants.

1. **FTP (port 21)** : détectez et affichez les commandes `USER` et `PASS` échangées en clair dans la session TCP. Associez les deux commandes pour afficher la paire identifiant / mot de passe reconstituée, accompagnée de l'adresse IP de la victime et de celle du serveur.
2. **HTTP (port 80)** : analysez les requêtes de méthode `POST` et extrayez les champs du corps dont le nom contient l'un des mots-clefs suivants : `password`,

`passwd`, `pwd`, `pass`, `login`, `username`, `user`, `email`. Affichez également l'URL ciblée par la requête.

3. **Telnet (port 23)** : reconstituez les caractères saisis par l'utilisateur en tenant compte du protocole de négociation Telnet (octets IAC). Affichez la séquence de caractères reconstituée permettant d'identifier les identifiants de connexion.

Pour chaque credential capturé, l'affichage doit inclure : l'horodatage, l'adresse IP de la victime, l'adresse IP du serveur, le protocole concerné et les données extraites.

1.4.1. Questions de compréhension — Partie B

1. Expliquez pourquoi l'utilisation de HTTPS (HTTP sur TLS) rend impossible l'extraction de credentials décrite dans ce TP, même si l'attaquant est présent sur le même segment réseau que la victime.
2. Le protocole FTP dispose d'une variante sécurisée nommée FTPS. Existe-t-il une autre alternative au FTP intégrant le chiffrement ? Comparez les deux en termes de mécanismes de sécurité et de compatibilité avec les infrastructures existantes.

Livrables attendus

- Code source commenté de `sniffer.py` incluant les modules d'extraction de la partie B.
- Capture d'écran de la console montrant au moins un credential FTP intercepté et un credential HTTP intercepté.
- Fichier `capture.pcap`.
- Réponses argumentées à l'ensemble des questions de compréhension.
- Paragraphe de conclusion de quinze à vingt lignes expliquant comment la généralisation de TLS et les bonnes pratiques de configuration réseau neutralisent ce type d'attaque passive.

2. TP2 — Empoisonnement DNS (DNS Spoofing)

2.1. Contexte

Le DNS (Domain Name System) est le service qui traduit les noms de domaine en adresses IP. Dans sa spécification originale, il ne propose aucun mécanisme d'authentification des réponses : n'importe quelle machine peut forger une réponse DNS en apparence valide si elle connaît l'identifiant de transaction de la requête correspondante.

Un attaquant capable d'intercepter les requêtes DNS ou de se positionner entre la victime et son résolveur peut répondre avant le serveur légitime en fournissant une adresse IP de son choix. Associée à un serveur de phishing, cette technique permet de rediriger silencieusement un utilisateur vers un service malveillant sans que l'URL affichée dans son navigateur ne change.

Objectifs pédagogiques

- Comprendre la structure d'un paquet DNS et les champs critiques d'une réponse de type A.
- Implémenter un outil d'analyse passive du trafic DNS.
- Forger des réponses DNS arbitraires et les injecter dans le flux réseau par programmation Python.
- Valider la redirection depuis la machine victime en situation réelle de laboratoire.
- Analyser les contremesures : DNSSEC, DNS over HTTPS, DNS over TLS.

2.2. Rappels protocolaires

Le tableau suivant récapitule les champs DNS importants pour ce TP.

Champ	Signification	Valeur attendue pour une réponse
qr	Question (0) ou Réponse (1)	1
aa	Authoritative Answer	1
id	Identifiant de transaction	Identique à la requête capturée
qd	Section Question	Copie de la question de la requête
an	Section Answer	Enregistrement avec l'IP forgée
qdcount	Nombre de questions	1
ancount	Nombre de réponses	1

Point critique : Pour que votre réponse forgée soit acceptée par le résolveur de la victime, l'identifiant de transaction DNS (`id`) de votre réponse doit impérativement correspondre à celui de la requête interceptée. Tout écart entraîne le rejet silencieux de la réponse forgée.

2.3. Partie A — Analyseur DNS passif

Travail demandé

Écrivez un script Python nommé `dns_analyse.py` qui surveille passivement le trafic DNS sur le réseau et produit les sorties suivantes.

1. Pour chaque requête DNS capturée (champ `qr == 0`), affichez : l'horodatage, l'adresse IP du client demandeur, le nom de domaine demandé et le type d'enregistrement sollicité (A, AAAA, MX, CNAME, etc.).
2. Pour chaque réponse DNS capturée (champ `qr == 1`), affichez : le nom de domaine résolu, l'adresse IP retournée et la valeur TTL de l'enregistrement.
3. Maintenez un compteur de fréquence par domaine demandé. Toutes les trente secondes, affichez les dix domaines les plus fréquemment sollicités sans interrompre la capture en cours.

2.4. Partie B — Implémentation de l'empoisonnement DNS

Travail demandé

Écrivez un script Python nommé `dns_spoof.py` qui intercepte les requêtes DNS et injecte des réponses forgées pour les domaines ciblés.

1. Le script accepte en ligne de commande les paramètres suivants :
 - Un ou plusieurs noms de domaine à intercepter.
 - Une adresse IP de redirection correspondant à votre serveur Web de démonstration.
 - Le nom de l'interface réseau à surveiller.
 - Un paramètre `-verbose` pour afficher le détail de chaque interception en console.
2. Pour chaque requête DNS de type A dont le domaine demandé figure dans la liste des cibles, le script construit et envoie une réponse UDP forgée contenant un enregistrement A pointant vers l'adresse IP spécifiée.
3. Les requêtes DNS portant sur des domaines non ciblés ne doivent subir aucune perturbation et doivent être traitées normalement par le résolveur légitime.
4. Avant d'envoyer une réponse forgée, le script vérifie que la requête est bien de type A (adresse IPv4) et ignore les autres types d'enregistrement (AAAA, MX, CNAME, etc.).

Préparation préalable : Pour que la machine victime achemine ses requêtes DNS vers votre machine attaquante, configurez manuellement l'adresse du serveur DNS de la victime avec l'IP 192.168.100.10 dans ses paramètres réseau, ou utilisez l'attaque

ARP du TP3 comme préalable.

2.5. Partie C — Validation et démonstration

Travail demandé

Réalisez la démonstration complète suivante et documentez chaque étape par une capture d'écran.

1. Lancez un serveur HTTP minimal sur votre machine attaquante (port 80) affichant une page de votre choix.
2. Depuis la machine victime, effectuez une résolution DNS manuelle du domaine cible en spécifiant explicitement votre machine comme serveur DNS. Vérifiez que l'adresse IP retournée correspond bien à celle de votre machine attaquante.
3. Depuis la machine victime, ouvrez un navigateur et naviguez vers le domaine cible en HTTP. Vérifiez que la page servie est bien celle de votre serveur local.
4. Depuis la machine victime, naviguez vers un domaine non ciblé et vérifiez que la navigation fonctionne normalement sans perturbation.

2.5.1. Questions de compréhension

1. Qu'est-ce que DNSSEC ? Décrivez le mécanisme par lequel cette extension invalide l'attaque réalisée dans ce TP. Pourquoi son adoption reste-t-elle limitée en pratique, notamment dans les environnements d'entreprise ?
2. DNS over HTTPS (DoH) et DNS over TLS (DoT) empêchent-ils l'attaque décrite dans ce TP ? Justifiez votre réponse en distinguant le cas où le résolveur du système est configuré en DoH/DoT de celui où seul le navigateur utilise DoH de façon autonome.
3. Quel champ du paquet DNS garantit que votre réponse forgée sera associée à la bonne requête par le résolveur de la victime ? Que se passerait-il si ce champ ne correspondait pas ?
4. Dans un environnement de production, quelles mesures côté commutateur ou pare-feu permettent de limiter ce type d'attaque sans nécessiter le déploiement de DNSSEC ?

Livrables attendus

- Code source commenté de `dns_analyse.py` et de `dns_spoof.py`.
- Captures d'écran documentant chaque étape de la partie C : console de l'attaquant, terminal de la victime et navigateur.
- Schéma du flux de l'attaque faisant apparaître les trois machines, les paquets échangés et le moment de l'injection de la réponse forgée.
- Réponses argumentées aux quatre questions de compréhension.

3. TP3 — ARP Spoofing et Redirection de Trafic avec iptables

3.1. Contexte

L'ARP (Address Resolution Protocol) permet d'associer une adresse IP à une adresse MAC sur un réseau local. Ce protocole est sans état et sans authentification : toute machine peut envoyer une réponse ARP non sollicitée, appelée *gratuitous ARP*, qui sera acceptée et stockée dans le cache ARP des autres machines du segment sans aucune vérification.

L'ARP Spoofing exploite cette absence d'authentification pour placer l'attaquant en position de Man-in-the-Middle entre la victime et la passerelle du réseau. En associant cette technique à la configuration du transfert IP Linux et des règles `iptables`, il devient possible de rediriger, journaliser ou analyser l'intégralité du trafic de la victime de façon transparente.

Ce TP est le plus complet des trois : il combine la manipulation de paquets bas niveau, la configuration système Linux et la compréhension des mécanismes de routage.

Objectifs pédagogiques

- Comprendre le protocole ARP, le fonctionnement du cache ARP et la notion de *gratuitous ARP*.
- Implémenter un empoisonneur ARP bidirectionnel en Python.
- Activer et vérifier le transfert de paquets IP sur Linux (IP forwarding).
- Utiliser `iptables` pour rediriger du trafic réseau de façon transparente vers un service local.
- Restaurer l'état original du cache ARP des machines cibles en fin d'attaque.
- Identifier les contremesures deployables : inspection ARP dynamique (DAI), VLAN, 802.1X.

3.2. Rappels protocolaires

3.2.1. Champs d'un paquet ARP

Champ	Signification	Valeur pour un Reply forgé
op	Opération : 1=Request, 2=Reply	2
psrc	Adresse IP source annoncée	IP à usurper
pdst	Adresse IP destination	IP de la machine cible
hwsrc	Adresse MAC source annoncée	MAC de la machine attaquante
hwdst	Adresse MAC destination	MAC réelle de la machine cible

3.2.2. Principe de l'attaque bidirectionnelle

Pour se placer en Man-in-the-Middle entre la victime V et la passerelle G, la machine attaquante A doit envoyer deux séries de paquets ARP Reply en continu.

- **Vers la victime V** : le paquet annonce que l'adresse IP de la passerelle G correspond désormais à la MAC de A.
- **Vers la passerelle G** : le paquet annonce que l'adresse IP de la victime V correspond désormais à la MAC de A.

Les deux machines mettent à jour leur cache ARP avec la MAC de l'attaquant. Tout le trafic entre V et G transite alors par A, qui doit le transférer pour maintenir la connectivité apparente de la victime.

3.3. Partie A — Relevé de l'état initial

Travail demandé

Avant toute manipulation, documentez l'état initial du cache ARP sur les trois machines du laboratoire.

1. Sur chaque machine, affichez le contenu du cache ARP et notez toutes les associations IP / MAC présentes.
2. Identifiez l'adresse MAC réelle de la passerelle telle qu'elle apparaît dans le cache ARP de la victime.
3. Identifiez l'adresse MAC réelle de la victime telle qu'elle apparaît dans le cache ARP de la passerelle.

Ces informations constituent la référence indispensable pour valider l'attaque et effectuer la restauration correcte en fin de TP.

3.4. Partie B — Empoisonneur ARP bidirectionnel

Travail demandé

Écrivez un script Python nommé `arp_spoof.py` réalisant l'empoisonnement ARP bidirectionnel et respectant les spécifications suivantes.

1. Le script accepte en ligne de commande : l'adresse IP de la victime, l'adresse IP de la passerelle et le nom de l'interface réseau à utiliser.
2. Au démarrage, le script résout automatiquement les adresses MAC correspondant aux deux IP fournies en émettant des requêtes ARP Request et en lisant les réponses reçues.
3. Le script envoie en boucle, toutes les deux secondes, les deux paquets ARP Reply nécessaires pour maintenir les caches empoisonnés : un vers la victime et un vers la passerelle.

4. En console, pour chaque envoi, le script affiche un message indiquant quelle fausse association IP / MAC est implantée dans quel cache.
5. Sur interruption clavier, le script restaure impérativement les caches ARP des deux machines cibles en envoyant des paquets ARP corrects avec les vraies adresses MAC, puis se termine proprement. Cette étape de restauration est obligatoire et évaluée.

3.5. Partie C — Activation du transfert IP

Travail demandé

Pour que le trafic de la victime soit bien acheminé vers la passerelle et non éliminé silencieusement, activez et vérifiez le transfert de paquets IP sur la machine attaquante.

1. Identifiez et appliquez la commande système permettant d'activer le forwarding IP de façon temporaire sur Linux, valable jusqu'au prochain redémarrage.
2. Identifiez la modification du fichier de configuration système permettant de rendre ce paramètre permanent au redémarrage.
3. Vérifiez que le forwarding est bien actif en lisant la valeur correspondante dans le pseudo-système de fichiers `/proc`.
4. Depuis la machine victime, vérifiez que la connectivité vers la passerelle est maintenue pendant que l'attaque ARP est active.

3.6. Partie D — Redirection de trafic avec iptables

Travail demandé

Une fois le Man-in-the-Middle établi et le forwarding IP actif, configurez `iptables` pour intercepter et rediriger le trafic HTTP de la victime.

1. Écrivez et appliquez la règle `iptables` de la table `nat`, chaîne `PREROUTING`, permettant de rediriger tout paquet TCP à destination du port 80 vers un port local de votre choix sur la machine attaquante.
2. Lancez un serveur HTTP minimal sur ce port local.
3. Depuis la machine victime, naviguez vers un site accessible en HTTP sur le réseau de laboratoire. Vérifiez que votre serveur local répond à la place du serveur distant.
4. Listez les règles `iptables` actives et commentez en détail la règle que vous venez d'ajouter, champ par champ : table, chaîne, protocole, port de destination, action et port de redirection.

5. En fin de TP, supprimez la règle `iptables` ajoutée et désactivez le forwarding IP. Vérifiez que les caches ARP des deux machines cibles sont correctement restaurés à leurs valeurs initiales relevées en partie A.

3.7. Partie E — Extension au contexte Wi-Fi

Travail demandé

Répondez par écrit aux questions suivantes en lien avec le déploiement de cette attaque sur un réseau Wi-Fi.

1. En quoi le contexte Wi-Fi diffère-t-il d'un réseau Ethernet commuté pour la mise en œuvre de l'ARP Spoofing ? Quels avantages ou contraintes supplémentaires ce contexte apporte-t-il à l'attaquant ?
2. Quelle interface devez-vous utiliser sur Kali Linux pour écouter le trafic sur un réseau Wi-Fi en mode moniteur ? Quelle commande permet de passer une carte Wi-Fi dans ce mode ?
3. Expliquez pourquoi un réseau Wi-Fi protégé par WPA2-Personal (clé partagée PSK) ne protège pas contre l'ARP Spoofing une fois que l'attaquant est authentifié sur ce même réseau.
4. Quelle fonctionnalité disponible sur les commutateurs gérés permet de bloquer l'ARP Spoofing au niveau de l'infrastructure réseau ? Décrivez son principe de fonctionnement en deux ou trois phrases.

3.7.1. Questions de compréhension générales

1. Quelle est la différence entre un ARP Request, un ARP Reply et un *gratuitous ARP* ? Lequel de ces trois types de paquets utilisez-vous dans votre script pour empoisonner les caches ?
2. Pourquoi est-il nécessaire d'empoisonner les caches ARP des deux machines (victime ET passerelle) et pas seulement celui de la victime ? Que se passerait-il si vous n'empoisonniez que le cache de la victime ?
3. Pourquoi la restauration du cache ARP en fin d'attaque est-elle une étape critique, aussi bien sur le plan technique que sur le plan éthique dans un contexte de test d'intrusion autorisé ?
4. Proposez deux architectures réseau ou configurations système distinctes qui rendraient cette attaque in-opérante ou significativement plus difficile à réaliser.

Livrables attendus

- Code source commenté de `arp_spoof.py` incluant la phase de restauration sur interruption clavier.
- Captures d'écran du cache ARP sur chaque machine : avant l'attaque, pendant l'attaque et après restauration.
- Capture d'écran montrant la redirection HTTP effective depuis le navigateur de la machine victime.
- Listing annoté de toutes les règles `iptables` appliquées.
- Réponses argumentées à toutes les questions de compréhension et aux questions de la partie Wi-Fi.
- Conclusion comparative de vingt à trente lignes mettant en regard les trois travaux pratiques et expliquant comment une architecture Zero Trust réduit la surface d'attaque exposée par ces trois techniques.